

Programación Orientada a Objetos – Sección 30
Henry Emanuel Vicente Alvarado – Carné 26275

1. ¿Qué clases son necesarias para representar el problema planteado?

OrdenServicio (Modelo): representa una orden individual.

GestorOrdenes (Modelo): administra la colección dinámica y las operaciones sobre ella.

VistaTaller (Vista): muestra menús y resultados al usuario.

ControladorTaller (Controlador): conecta la vista con el gestor, captura entradas y maneja excepciones.

Main (Driver): punto de entrada.

Punto 2, 3 6 y 7. ¿Qué propiedades y métodos tendrá cada clase? ¿Qué tipo deben tener las propiedades y métodos de cada clase? ¿Cuáles deben ser los modificadores de visibilidad de los miembros de cada clase? ¿Qué parámetros serán requeridos por los métodos de sus clases?

Clase	Atributo	Tipo	Visibilidad	Propósito
OrdenServicio	numeroOrden	int	private	Identifica cada orden de servicio
	nombrePropietario	String	private	Almacena el nombre del dueño del vehículo
	placa	String	private	Almacena la placa del vehículo asociado a la orden
	descripcionServicio	String	private	Describe el trabajo o mantenimiento a realizar
	costoEstimado	double	private	Guarda el costo estimado del servicio
GestorOrdenes	ordenes	List<OrdenServicio>	private	Colección dinámica que almacena todas

Programación Orientada a Objetos – Sección 30
Henry Emanuel Vicente Alvarado – Carné 26275

				las órdenes activas
VistaTaller	(Scanner)	Scanner	private	Captura la entrada del usuario desde consola
ControladorTaller	gestor	GestorOrdenes	private	Referencia al modelo para delegar la lógica de negocio
ControladorTaller	vista	VistaTaller	private	Referencia a la vista para mostrar menús y resultados

Clase	Método	Parámetros	Return	Visibilidad	Propósito
OrdenServicio	OrdenServicio	int numeroOrden, String nombrePropietario, String placa, String descripcionServicio, double costoEstimado		public	Crea una nueva orden con sus datos iniciales
	getNumeroOrden		int	public	Retorna el número de orden
	getNombrePropietario		String	public	Retorna el nombre del propietario
	getPlaca		String	public	Retorna la placa del vehículo

Programación Orientada a Objetos – Sección 30
Henry Emanuel Vicente Alvarado – Carné 26275

	getDescripcionServicio		String	public	Retorna la descripción del servicio
	setDescripcionServicio	String descripcion	void	public	Permite modificar la descripción del servicio
	getCostoEstimado		double	public	Retorna el costo estimado
	setCostoEstimado	double costo	void	public	Permite modificar el costo estimado
	toString		String	public	Devuelve una representación legible de la orden
GestorOrdenes	GestorOrdenes			public	Inicializa la colección dinámica de órdenes
	registrarOrden	OrdenServicio orden	void	public	Valida y agrega una nueva orden a la colección
	buscarPorNumero	int numeroOrden	OrdenServicio	public	Localiza una orden específica por su número

Programación Orientada a Objetos – Sección 30
Henry Emanuel Vicente Alvarado – Carné 26275

	modificarOrden	int numeroOrden, String nuevaDescripci on, double nuevoCosto	void	public	Actualiza descripci n y costo de una orden existente
	cancelarOrden	int numeroOrden	void	public	Elimina una orden de la colección dinámica
	consultarPorPlaca	String placa	List<OrdenServ icio>	public	Retorna todas las órdenes asociadas a una placa
	consultarTodas		List<OrdenServ icio>	public	Retorna todas las órdenes registradas actualment e
	calcularTotal		double	public	Suma los costos estimados de todas las órdenes activas
	calcularPromedio		double	public	Calcula el costo promedio de las órdenes activas
	obtenerMayorCosto		OrdenServicio	public	Determina la orden con el costo

Programación Orientada a Objetos – Sección 30
Henry Emanuel Vicente Alvarado – Carné 26275

					estimado más alto
	cantidadOrdenes		int	public	Indica cuántas órdenes hay registradas actualmente
	existeNumeroOrden	int numeroOrden	boolean	private	Verifica internamente si un número ya está registrado
VistaTaller	mostrarMenu		void	public	Despliega las opciones del menú principal
	leerOpcion		int	public	Captura la opción elegida por el usuario
	solicitarDatosOrden		OrdenServicio	public	Pide al usuario los datos para crear una orden
	mostrarOrden	OrdenServicio orden	void	public	Muestra en pantalla la información de una orden
	mostrarListaOrdenes	List<OrdenServicio> ordenes	void	public	Muestra en pantalla

Programación Orientada a Objetos – Sección 30
Henry Emanuel Vicente Alvarado – Carné 26275

					una lista de órdenes
	mostrarMensaje	String mensaje	void	public	Muestra mensajes generales o de error al usuario
Controlador Taller	ControladorTaller (constructor)			public	Inicializa el gestor y la vista
	iniciar		void	public	Ejecuta el ciclo principal del menú
	procesarRegistro		void	private	Coordina la captura y validación de una nueva orden
	procesarConsulta		void	private	Coordina la consulta de todas las órdenes
	procesarBusqueda		void	private	Coordina la búsqueda de una orden por número
	procesarModificaci on		void	private	Coordina la modificaci ón de una orden existente

Programación Orientada a Objetos – Sección 30
Henry Emanuel Vicente Alvarado – Carné 26275

	procesarCancelacion		void	private	Coordina la cancelación de una orden
	procesarConsultaPorPlaca		void	private	Coordina la consulta de órdenes por placa
	procesarReporteCostos		void	private	Coordina el cálculo y despliegue del total y promedio
	procesarMayorCosto		void	private	Coordina la búsqueda de la orden de mayor costo
	procesarCantidadOrdenes		void	private	Coordina el conteo de órdenes registradas
Main	main	String[] args	void	public static	

Programación Orientada a Objetos – Sección 30

Henry Emanuel Vicente Alvarado – Carné 26275

4. ¿Qué información deberá almacenarse dentro de la colección dinámica?

Objetos OrdenServicio completos, uno por cada orden registrada.

5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?

List<OrdenServicio> se usa como tipo declarado (atributo, parámetros y retornos de métodos, por ejemplo el resultado de consultarPorPlaca), y ArrayList<>() se usa solo en el momento de instanciar esa lista dentro del constructor de GestorOrdenes.

8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

Se asignan mediante el constructor de OrdenServicio, exigiendo que el controlador valide antes de construir el objeto (nombre y placa no vacíos, costo > 0). No hay valores por defecto "vacíos": si el usuario no da datos válidos, no se crea la orden.

9. ¿Cómo identificará una orden específica dentro de la colección?

Por su numeroOrden, que debe ser único. La búsqueda recorre la lista comparando ese campo.

10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?

Agregar ordenes.add(nuevaOrden) tras validar que el número no exista. Eliminar: localizar el objeto con buscarPorNumero y usar ordenes.remove(orden) (o removeIf(o -> o.getNumeroOrden() == numero)); si no se encuentra, se lanza/maneja una excepción.

11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

Recorrer la lista con un for-each, comparar orden.getPlaca().equals(placa) y agregar cada coincidencia a una lista de resultados (List<OrdenServicio>) que se retorna a la vista.

12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

Un for-each suma costoEstimado de cada orden en un acumulador double total. El promedio es total / ordenes.size(), cuidando dividir entre cero si la lista está vacía.

Programación Orientada a Objetos – Sección 30

Henry Emanuel Vicente Alvarado – Carné 26275

13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?

Recorrer la lista comparando el costo de cada orden contra un máximo acumulado, guardando la referencia al objeto cuando se encuentra uno mayor.

14. ¿Qué situaciones dentro del programa pueden producir una excepción?

Costo ingresado no numérico (NumberFormatException al parsear), buscar, modificar o cancelar un número de orden que no existe e intentar registrar un número de orden ya existente.

15. ¿Qué operaciones serán protegidas utilizando try-catch?

Registrar orden, buscar orden, modificar orden y cancelar orden.

16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse

independientemente del resultado de la operación?

La operación de búsqueda o modificación, el finally puede mostrar un mensaje como "Operación de búsqueda finalizada" y devolver el control al menú principal, ejecutándose sin importar si la orden existió o se lanzó una excepción.